

# Explicação linha por linha

## 1. Classe

```
class MyApp extends StatelessWidget {
```

`MyApp` é o widget principal da aplicação.

Ela herda de `StatelessWidget`, ou seja, ela **não possui estado próprio**.

---

## 2. Construtor

```
const MyApp({super.key});
```

Recebe uma `Key` para identificação do widget.

O `const` permite que o Flutter reutilize esse widget quando possível, melhorando o desempenho.

---

## 3. Método build

```
Widget build(BuildContext context)
```

Todo widget precisa implementar o método `build()`.

Ele retorna os widgets que serão desenhados na tela.

---

## 4. MaterialApp

```
return MaterialApp(
```

É o widget raiz de aplicações que seguem o Material Design.

Ele configura:

- navegação
- tema
- rotas
- título
- tela inicial

---

## 5. Título

title: "Aprendendo BLoC",

É o nome da aplicação.

---

## 6. Remover a faixa DEBUG

debugShowCheckedModeBanner: false,

Remove aquela faixa vermelha "DEBUG".

---

## 7. MultiBlocProvider

home: MultiBlocProvider(

Aqui começa a parte do BLoC.

O **MultiBlocProvider** serve para disponibilizar **vários BLoCs** para os widgets abaixo dele.

Hoje você tem apenas um.

Amanhã pode ter:

- CounterBloc
- LoginBloc
- ThemeBloc
- UserBloc

Tudo no mesmo lugar.

---

## 8. providers

providers: [

É uma lista de provedores.

No seu código estava:

providers: BlocProvider(

Mas `providers` espera uma **lista**, por isso precisa usar `[]`.

---

## 9. BlocProvider

`BlocProvider`(

Ele cria um BLoC e o disponibiliza para todos os widgets filhos.

---

## 10. create

`create: (context) => CounterBloc(),`

Sempre que a aplicação iniciar, será criado um:

`CounterBloc()`

Esse objeto ficará disponível para a árvore inteira.

---

## 11. child

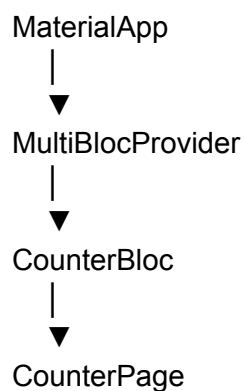
`child: const CounterPage(),`

É a tela que utilizará esse BLoC.

Como o `CounterPage` está abaixo do `BlocProvider`, ele consegue acessar o `CounterBloc`.

---

## Como a árvore fica



A **CounterPage** consegue fazer:

```
context.read<CounterBloc>()
```

porque o **CounterBloc** foi colocado acima dela.

---

## Por que usar MultiBlocProvider?

Hoje:

CounterBloc

Amanhã:

CounterBloc

LoginBloc

ThemeBloc

UserBloc

CartBloc

Sem **MultiBlocProvider** ficaria assim:

```
BlocProvider(  
  create: (_) => CounterBloc(),  
  child: BlocProvider(  
    create: (_) => LoginBloc(),  
    child: BlocProvider(  
      create: (_) => ThemeBloc(),  
      child: CounterPage(),  
    ),  
  ),  
);
```

Com **MultiBlocProvider**, fica muito mais organizado:

```
MultiBlocProvider(  
  providers: [  
    BlocProvider(create: (_) => CounterBloc()),  
    BlocProvider(create: (_) => LoginBloc()),  
    BlocProvider(create: (_) => ThemeBloc()),  
  ],  
  child: CounterPage(),  
);
```

---

## Erros no seu código

Você escreveu:

```
providers: BlocProvider(
```

O correto é:

```
providers: [  
  BlocProvider(  
    create: (_) => CounterBloc(),  
  ),  
],
```

Porque `providers` recebe uma **lista** de `BlocProviders`, mesmo que ela tenha apenas um elemento.

**Resumo:** `MultiBlocProvider` é um "container" para disponibilizar um ou mais BLoCs na árvore de widgets. Cada `BlocProvider` cria uma instância de um BLoC, e qualquer widget abaixo dele pode acessá-la usando `context.read<T>()`, `context.watch<T>()` ou widgets como `BlocBuilder` e `BlocListener`.